

Documentation: RNN Channel Equalizer

By Roshan Sharma (2204229)

Module Descriptions

A. Arithmetic and Activation

These modules constitute the fundamental mathematical processing units utilized throughout the hierarchy.

- **multiplier, adder, subtractor (FP32)**
 - **Functional Description:** Implementing IEEE-754 compliant single-precision floating-point arithmetic operations.
 - **Application:** Instantiated within higher-level modules to execute matrix-vector multiplications and gradient descent updates.
- **dot_product**
 - **Functional Description:** Computes the scalar product of two input vectors
 - **Design Optimization:** Utilizes a sequential Finite State Machine (FSM) to drive a single multiplier-accumulator pair, processing vector elements iteratively. This significantly reduces usage compared to parallel implementation.
- **tanh / SigMoid**
 - **Functional Description:** Approximates non-linear activation functions essential for GRU gating logic.
 - **Design Optimization:** Implements a **Look-Up Table (LUT)** architecture with 4096 entries. A custom address decoding logic inspects the IEEE exponent field to identify saturation regions, mapping only the active non-linear range of the function to physical memory.
- **gradient_clipper**
 - **Functional Description:** Enforces a magnitude constraints on gradient values (e.g., $= \pm 1.0$) to mitigate the "Exploding Gradient" phenomenon prevalent in RNN training.
 - **Design Optimization:** Performs magnitude comparison using integer-based logic on the raw floating-point bit fields, ensuring single-cycle latency without requiring floating-point comparators.

B. Inference

These modules are responsible for the input-to-output data flow, calculating the network prediction.

- **gate_cal**
 - **Functional Description:** The primary Arithmetic Logic Unit (ALU) for the GRU. It computes the activation equations.
 - **Application:** Used to calculate the Reset Gate), Update Gate, and the Candidate Hidden State.
- **GRU_Cell_BPTT**
 - **Functional Description:** Orchestrates the computational sequence for a single GRU neuron.
 - **Application:** Implements a two-phase FSM. Phase 0 computes the Reset gate; Phase 1 computes the Update gate and the final new state equation.
- **GRU_Layer_BPTT**
 - **Functional Description:** Manages the computation for the complete hidden layer (comprising 3 units).
 - **Application:** Implements serialization of the GRU_Cell. It iterates through neuron indices (0 to 2), reusing the single cell instance. Crucially, it buffers intermediate states into local cache memory, which is a prerequisite for the subsequent BPTT process.
- **linear_layer**
 - **Functional Description:** The fully connected output layer mapping the Hidden State vector to the final QPSK prediction (In-Phase and Quadrature components).

C. Training

These modules implement the multivariate Chain Rule to derive gradients for weight optimization.

- **mse_loss**
 - **Functional Description:** Computes the Mean Squared Error (MSE) objective function and derives the initial error gradient relative to the output.
- **linear_layer_backward**
 - **Functional Description:** Propagates the error gradient from the Output Layer back to the Hidden Layer inputs.
- **GRU_Backward**
 - **Functional Description:** The core calculus engine implementing BPTT.
 - **Application:** It accepts the gradient from the subsequent timestep ($t+1$) and the cached forward states. It computes partial derivatives for all weight matrices (W , U , b) and derives the error gradient to be propagated to the previous timestep ($t-1$).

D. Control

- **Training_Controller**
 - **Functional Description:** The master controller managing the temporal sequence of operations.
 - **Application:**
 - **Forward Phase:** Feeds input sequences to the network and triggers state snapshotting to Cache Memory.
 - **Backward Phase:** Iterates time backwards (T to 0), retrieves cached, and drives the GRU_Backward module.
 - **Batch Management:** signals weight updates only upon the completion of a defined BATCH_SIZE.
- **GRU_top**
 - **Functional Description:** The top-level module housing the physical weight registers and the Stochastic Gradient Descent (SGD) engine.
 - **Application:**
 - **Initialization:** Utilizes a Linear Feedback Shift Register (LFSR) to initialize weights with pseudo-random floating-point values.
 - **Update Engine:** Employs a multiplexing scheme to iterate through all parameter sets, updating them using a shared arithmetic pipeline via the SGD rule: $W_{\text{new}} = W_{\text{old}} - lr * \text{gradient}$
- **weight_updater_single / gradient_accumulator_single**
 - **Functional Description:** Wrappers for arithmetic cores that enforce a handshake protocol, preventing race conditions during register updates.

E. FPGA Interface

- **fpga_top_module**
 - **Functional Description:** The physical abstraction layer for the DE10-Lite FPGA board.
 - **Application:** Interacts with on-board peripherals including the ROM (Dataset), PLL (Clock Generation), and I/O interfaces (Switches/LEDs).

Parameters

1. BATCH_SIZE

- **Description:** The number of samples accumulated before a weight update occurs.
- **Modification:**
 - **Smaller Batch:** Faster updates, more noisy convergence (stochastic).
 - **Larger Batch:** More stable gradients, slower updates.

2. SAMPLES_PER_EPOCH (Default: 3500).

- **Description:** Defines the cycle length for logging and epoch counting.
- **Modification:** Match this to the size of the dataset stored in the ROM to ensure accurate epoch tracking.

3. CLIP_VAL (Default: 1.0)

- **Description:** The threshold for the `gradient_clipper` module.
- **Modification:** Adjusting this hex value changes the gradient clamping limit, which acts as a hyperparameter for training stability.

4. INPUT_FEATURES (Default: 2)

- **Description:** Defines the dimensionality of the input vector.
- **Modification:**
 - Set to 1 for Real-valued signals (e.g., BPSK).
 - Set to 2 for Complex-valued signals (e.g., QPSK, I/Q data).

5. GRU_UNITS (Default: 3)

- **Description:** Defines the size of the hidden state vector (number of neurons).
- **Modification:** Increasing this value improves the model's capacity to learn complex non-linear distortions but increases FPGA resource usage linearly.

6. OUTPUT_SIZE (Default: 2)

- **Description:** Defines the size of the final prediction vector.
- **Modification:**
 - Set to 1 for binary classification.
 - Set to 2 for I/Q symbol prediction.

7. SEQUENCE_LENGTH (Default: 3)

- **Description:** Defines the BPTT unrolling depth (time steps).
- **Modification:** Increase this parameter to capture longer-term Inter-Symbol Interference (ISI) in highly dispersive channels. Note that BRAM usage for caching scales linearly with this parameter.

Exported weights from Python model (for inference)

Wr (Reset Input)

```
Wr_flat[0*32 +: 32] = 32'hdbbb8b27;  
Wr_flat[1*32 +: 32] = 32'h3ee5fecb;  
Wr_flat[2*32 +: 32] = 32'h3e8b0b17;  
Wr_flat[3*32 +: 32] = 32'hbe0b83a1;  
Wr_flat[4*32 +: 32] = 32'hbf465abc;  
Wr_flat[5*32 +: 32] = 32'hbf89c6a3;
```

Ur (Reset hidden)

```
Ur_flat[0*32 +: 32] = 32'hbe8894dd;  
Ur_flat[1*32 +: 32] = 32'hbf063418;  
Ur_flat[2*32 +: 32] = 32'h3f4b5add;  
Ur_flat[3*32 +: 32] = 32'hbf2dbf62;  
Ur_flat[4*32 +: 32] = 32'h3e8d4890;  
Ur_flat[5*32 +: 32] = 32'h3fb7dd50;  
Ur_flat[6*32 +: 32] = 32'hbdf13aa2;  
Ur_flat[7*32 +: 32] = 32'hbf1efec5;  
Ur_flat[8*32 +: 32] = 32'hbedcae16;
```

br (Reset Bias)

```
br_flat[0*32 +: 32] = 32'h3fbee325;  
br_flat[1*32 +: 32] = 32'h3f1921fa;  
br_flat[2*32 +: 32] = 32'h3f38ef97;
```

Wz (Update Input)

```
Wz_flat[0*32 +: 32] = 32'hbf3ac95e;  
Wz_flat[1*32 +: 32] = 32'h3f0648a8;  
Wz_flat[2*32 +: 32] = 32'h3d142a72;  
Wz_flat[3*32 +: 32] = 32'hbd5348fe;
```

Wz_flat[4*32 +: 32] = 32'h3eb35010;

Wz_flat[5*32 +: 32] = 32'h3e87ccf2;

Uz (Update hidden)

Uz_flat[0*32 +: 32] = 32'h3efdea76;

Uz_flat[1*32 +: 32] = 32'h3ec47954;

Uz_flat[2*32 +: 32] = 32'hbe6de2af;

Uz_flat[3*32 +: 32] = 32'h3f297d18;

Uz_flat[4*32 +: 32] = 32'h3e201356;

Uz_flat[5*32 +: 32] = 32'hbf98501a;

Uz_flat[6*32 +: 32] = 32'h3f7c3779;

Uz_flat[7*32 +: 32] = 32'hbfb94f40;

Uz_flat[8*32 +: 32] = 32'hc07752e1;

bz (Update Bias)

bz_flat[0*32 +: 32] = 32'hbfc6783d;

bz_flat[1*32 +: 32] = 32'hbfc72bf9;

bz_flat[2*32 +: 32] = 32'hbfca04b4;

--- Wh (New Input)

Wh_flat[0*32 +: 32] = 32'h3df5a05f;

Wh_flat[1*32 +: 32] = 32'hbf3c972b;

Wh_flat[2*32 +: 32] = 32'h3ddc4e13;

Wh_flat[3*32 +: 32] = 32'h3c4c9b12;

Wh_flat[4*32 +: 32] = 32'h3ee68962;

Wh_flat[5*32 +: 32] = 32'h3f286cc9;

--- Uh (New hidden)

Uh_flat[0*32 +: 32] = 32'h40297973;

Uh_flat[1*32 +: 32] = 32'h3f87460f;

Uh_flat[2*32 +: 32] = 32'hbfbc5cac;

```
Uh_flat[3*32 +: 32] = 32'hc05439c3;  
Uh_flat[4*32 +: 32] = 32'h3fac427c;  
Uh_flat[5*32 +: 32] = 32'h3f743033;  
Uh_flat[6*32 +: 32] = 32'h3f7ed536;  
Uh_flat[7*32 +: 32] = 32'hc05362b6;  
Uh_flat[8*32 +: 32] = 32'h406880e2;
```

bh (New Bias)

```
bh_flat[0*32 +: 32] = 32'hbe46cf47;  
bh_flat[1*32 +: 32] = 32'hbf45e120;  
bh_flat[2*32 +: 32] = 32'h3ef4754a;
```

FC Weights (linear layer)

```
fc_weights_flat[0*32 +: 32] = 32'h3c5c3f21;  
fc_weights_flat[1*32 +: 32] = 32'h3f87ef7e;  
fc_weights_flat[2*32 +: 32] = 32'h3d2b8d7a;  
fc_weights_flat[3*32 +: 32] = 32'hbceb175f;  
fc_weights_flat[4*32 +: 32] = 32'hbcfacfa5;  
fc_weights_flat[5*32 +: 32] = 32'hbf95cfd6;
```

FC Bias

```
fc_bias_flat[0*32 +: 32] = 32'hbb1dc645;  
fc_bias_flat[1*32 +: 32] = 32'h3dfe837d;
```